Matrix Transposition, Cache Resonance, and a Compiler Surprise

Side-lecture notes — Foundations of High Performance Computing

A bit of context
These notes come from a debugging session I had while preparing the exercises on cache optimisation.

I was putting together an example on how to write an adaptive cache-friendly algorithm (the tiling requires the tuning of the block size, as we have seen). The two codes I had in mind were: (A) the traversing by Z-order, and (B) a "divide-et-impera" approach which reproduces the same pattern.

I wanted to highlighting that, while elegant and effective in terms of cache usage, the Z-order traversing comes with a large overhead in terms of instructions due to the bit extraction, and that a simple strategy like the divide et impera is in practice more effective.

Reasoning on the codes and the obtained result, I also realize that due to its limitation (in pragmatic terms) to power-of-two sizes, the Z-order algorithm suffers for a much worse limitation: it is a clean demonstration of cache set conflicts, which we already saw when introducing the matrix transposition problem.

Instead, I stumbled into something I had not planned for at all, and that turned out to be a wonderful case to discuss the memory aliasing in C.
I am writing it up because the whole story, accident included, turned out to be more instructive than the original exercise.

---

1. The transposition problem

Let me summarize the problem. Matrix transposition is almost offensively simple. Given an $N \times N$ matrix A, produce B such that $B[j][i] = A[i][j]$. It is three lines of code:

```
for (int i = 0; i < N; i++)
    for (int j = 0; j < N; j++)
        B[j*N + i] = A[i*N + j];
```

No arithmetic at all — one load and one store per element, $O(N^2)$, end of story. And yet, for large matrices, this trivial operation runs painfully slow; much worse than what the memory bandwidth of the machine would allow. The problem is entirely in the access pattern, and in how it clashes with the cache.

In C row-major storage, `A[i*N + j]` with `j` varying in the inner loop gives you a perfect sequential stride-1 access. The prefetcher is happy, every cache line is fully consumed: 64-byte lines, 8-byte doubles, 8 useful elements per line, so one miss every 8 accesses — a 12.5% miss rate, which is fine (actually unavoidable).

The write side is another matter. `B[j*N + i]` with `j` running in the inner loop touches elements that are N doubles apart, i.e. $8N$ bytes. For a 2048×2048 matrix of doubles that means 16,384 bytes between consecutive stores. Each cache line fetched for writing serves *one* element and is then abandoned; the next iteration jumps somewhere else entirely. The spatial locality is zero. Every store is a miss.
However, I re-direct you to the considerations about the opportunity of having contiguous write instead of contiguous read due to the most-often implemented write-back cache policy.

A rough estimate of the damage: if the L1 hit costs ~4 cycles and the miss penalty is ~30 cycles (and it can be much worse if you go all the way to DRAM), the read stream is fine but the write stream pays 30 cycles on every element. That gives you about 30 cycles per element, versus 1–2 for a cache-friendly version. An order of magnitude, for a kernel that does not even touch a floating-point unit.

---

2. Cache-friendly approaches

The idea behind every cache-friendly transpose is always the same: re-organise the traversal so that at any given moment, both source and destination fall within a small region that fits in cache.

We have already illustrated the classical solution of transposition by tiling (which will be used also for matrix multiplication).

However, I want to examine two different strategies.

2.1. Divide et impera (recursive decomposition)

This is, in my view, the cleanest approach. You recursively cut the matrix in half along the longest dimension until the sub-blocks are small enough to fit in L1, and then you transpose each block with the naïve loop. At that point it does not matter which stream is strided — everything is in cache.

The base case does the actual element-wise copy on a small block:

```
void transpose_block(const dtype *src, dtype *dst,
                     int r_start, int r_end,
                     int c_start, int c_end,
                     int n, int m)
{
    // src is n × m, dst is m × n
    // outer loop on columns → writes are contiguous
    for (int j = c_start; j < c_end; j++)
        for (int i = r_start; i < r_end; i++)
            dst[j * n + i] = src[i * m + j];
}
```

A word on the loop order. The outer loop runs on `j`, which makes the writes to `dst[j*n + i]` sequential in `i` — contiguous in memory. The reads from `src[i*m + j]` are strided, but within a block that fits in cache this is fine. There is an alternative variant with the loops swapped, which makes reads contiguous and writes strided; which one wins depends on the write-allocate policy of the particular cache you are running on. In my experience on x86_64 Intel and AMD high-end computational cores, contiguous writes tend to be slightly better, but honestly it is the kind of thing you should just try.

Two details that will matter later. First, `src` is `const` — not just good practice, but a concrete hint to the compiler that the source is never modified through this pointer. Second, `src` and `dst` come from separate `malloc` calls in `main`. Keep both of these in mind; they become relevant in Section 7.

The recursive wrapper:

```
void transpose_cache_oblivious(const dtype *src, dtype *dst,
                               int r_start, int r_end,
                               int c_start, int c_end,
```

```
                                int n, int m, int threshold)
{
    int r_len = r_end - r_start;
    int c_len = c_end - c_start;

    if (r_len <= threshold && c_len <= threshold) {
        transpose_block(src, dst, r_start, r_end, c_start, c_end, n, m);
    }
    else if (r_len >= c_len) {
        int r_mid = r_start + (r_len >> 1);
        transpose_cache_oblivious(src, dst, r_start, r_mid,
                                  c_start, c_end, n, m, threshold);
        transpose_cache_oblivious(src, dst, r_mid, r_end,
                                  c_start, c_end, n, m, threshold);
    }
    else {
        int c_mid = c_start + (c_len >> 1);
        transpose_cache_oblivious(src, dst, r_start, r_end,
                                  c_start, c_mid, n, m, threshold);
        transpose_cache_oblivious(src, dst, r_start, r_end,
                                  c_mid, c_end, n, m, threshold);
    }
}
```

Always bisecting the longer dimension keeps the sub-blocks roughly square, which matters: a tall thin block has bad locality in one direction no matter what. The `threshold` controls when to stop recursing. A value of 64 means base-case blocks of up to 64×64 elements — that is $32\,\mathrm{KB}$ of doubles, about one L1. In the purist cache-oblivious formulation you would recurse down to 1×1, and the algorithm would adapt to any cache size automatically; in practice the function-call overhead of N^2 recursive invocations is unbearable, so we compromise.

The algorithm works for non-square matrices without modification. The classic explicit tiling (two extra nested loops over block indices) achieves the same cache behaviour with simpler code, but it requires choosing a tile size — a magic number tied to the hardware. Different machine, different optimal tile. The recursive approach is more robust to hardware variation, though the threshold plays a somewhat similar role, to be fair.

2.2. Z-order (Morton) traversal

There is a more unusual idea, which I find rather elegant. Instead of explicit recursion or tiling, you traverse the matrix along a space-filling curve — the Morton curve (Z-order curve) — which naturally visits the elements in a blocked pattern.

The Morton index of position (x, y) is constructed by interleaving the bits of x and y:

$$\mathrm{morton}(x, y) = \cdots y_2 x_2\, y_1 x_1\, y_0 x_0$$

The crucial property: elements close in Morton order are close in 2D space, in *both* the x and y directions simultaneously. A linear scan through Morton indices visits the matrix in progressively larger Z-shaped blocks — 2×2, 4×4, 8×8, and so on. It is essentially the recursion from Section 2.1 encoded directly in the index itself, without any function calls or stack frames.

For the transpose, we iterate a counter from 0 to $N^2 - 1$, deinterleave the bits to get (x, y), and copy. The bit-deinterleaving is done with a standard compaction trick:

```c
typedef struct { uint32_t x, y; } coord_t;

static inline coord_t compact_bits(uint32_t a)
{
    uint32_t b = a >> 1;
    a &= 0x55555555;                  // even bits
    b &= 0x55555555;                  // odd bits

    a = (a ^ (a >> 1)) & 0x33333333;
    b = (b ^ (b >> 1)) & 0x33333333;

    a = (a ^ (a >> 2)) & 0x0f0f0f0f;
    b = (b ^ (b >> 2)) & 0x0f0f0f0f;

    a = (a ^ (a >> 4)) & 0x00ff00ff;
    b = (b ^ (b >> 4)) & 0x00ff00ff;

    a = (a ^ (a >> 8)) & 0x0000ffff;
    b = (b ^ (b >> 8)) & 0x0000ffff;

    return (coord_t){.x = a, .y = b};
}
```

Each stage gathers every other bit into contiguous positions, progressively halving the spread. The transpose loop is then pleasantly flat:

```c
for (uint32_t counter = 0; counter < size; counter++)
{
    coord_t c = compact_bits(counter);
    uint32_t src_pos = c.x * k + c.y;
    uint32_t dst_pos = c.y * k + c.x;
    dst[dst_pos] = src[src_pos];
}
```

No recursion, no nested loops, no tile parameters. One flat sweep. In theory, the miss rate should approach 12.5% per stream (one miss per cache line of 8 doubles).

Nice idea. But there are complications, more than I thought at first sight.

---

3. Version 1: the power-of-two trap

My first Z-order implementation (file `matrix_transpose_z-order.v1.c`) allocates both matrices from a single contiguous block:

```c
uint32_t size = k * k;
dtype *src = (dtype*)malloc(sizeof(dtype) * size * 2);
dtype *dst = src + size;
```

Clean, one allocation, no waste and a bit of laziness. I compiled with `gcc -O3 -march=native -mtune=native`, ran on a 2048×2048 matrix of doubles, and measured the L1 miss rates with PAPI.

They were bad. About 25% for loads, 19% for stores — far from the 12.5% I expected.

Cache resonance

I like to call this *cache resonance*, although the proper textbook name is "set aliasing" or "set conflict". The word resonance captures what is happening physically: the data stride is in resonance with the cache geometry, and all accesses pile up in the same place.

On the test machine I used (i7-1360P, Golden Cove P-cores on my laptop), the L1 data cache is 48 KB, 12-way set-associative, 64-byte lines:

$$\text{sets} = \frac{48 \times 1024}{12 \times 64} = 64 \text{ sets}$$

Set index: bits 6–11 of the address. With $k = 2048$ and 8-byte doubles, the row stride is $2048 \times 8 = 16384 = 2^{14}$ bytes. The set-index difference between adjacent-row elements in the same column:

$$\frac{16384}{64} \bmod 64 = 256 \bmod 64 = 0$$

Zero. Every row maps to the same cache set. When the Z-order traversal works on an 8×8 block, it needs lines from 8 source rows and 8 destination rows — 16 lines, all wanting the same cache set, which has only 12 ways. Evictions are guaranteed; the set is oversubscribed.

And it gets worse. Since `dst = src + size` and `size = k^2 = $2^{22}$`, the offset between corresponding `src` and `dst` elements is 2^{25} bytes. Since $2^{25} \bmod (64 \times 64) = 0$, the `src` and `dst` elements fall in the exact same set — so the conflict pressure is doubled.

When the stride is a multiple of the number of sets (times the line size), every access collapses onto a handful of sets, the effective associativity drops to nothing, and the cache is essentially switched off. For power-of-two matrix dimensions this happens systematically.
It is one of those things that everybody who works seriously with dense arrays has been bitten by at least once.
Now, we can add to the list those working on HPC examples.

---

4. PAPI instrumentation

To diagnose this properly I needed hardware counters, not guesswork. I used PAPI, with a set of convenience macros in a header file (`mypapi.h`, see Appendix A). The important choice is the events:

```
int papi_events[] = {
    PAPI_TOT_INS,    // total retired instructions
    PAPI_TOT_CYC,    // total cycles
    PAPI_L1_LDM,     // L1 load misses
    PAPI_L1_STM,     // L1 store misses
    PAPI_LD_INS,     // load instructions
    PAPI_SR_INS      // store instructions
};
```

An important point which I want to stress. The generic `PAPI_L1_DCM` lumps together load and store misses. If you then divide that by `PAPI_LD_INS` to compute a "miss rate", you are putting store misses in the numerator and only load instructions in the denominator — the ratio makes no sense. You must keep them separate: `L1_LDM / LD_INS` for loads, `L1_STM / SR_INS` for stores. It sounds obvious, but I have seen people get this wrong more than once. Including myself, the first time.

The instrumented loop wraps the transpose in a start/stop pair that accumulates over 5 repetitions:

```
for (int rep = 0; rep < NREPETITIONS; rep++) {
    PAPI_START_CNTR;
    for (uint32_t counter = 0; counter < size; counter++) {
        coord_t c = compact_bits(counter);
        uint32_t src_pos = c.x * k + c.y;
        uint32_t dst_pos = c.y * k + c.x;
        dst[dst_pos] = src[src_pos];
    }
    PAPI_STOP_CNTR;
}
```

One practical note: the i7-1360P is a hybrid CPU with P-cores (Golden Cove) and E-cores (Gracemont). Different cache geometry, different counter semantics. If the OS migrates you between core types during the measurement, you get garbage. Pin the process: `taskset -c $ID_of_a_P_core ./transpose 11`.

---

5. Version 2: padding

The fix

Standard remedy for cache resonance: pad the leading dimension so the stride is not a multiple of the number of sets.
In the file `matrix_transpose_z-order.v2.c` you find the following:

```
uint32_t k_pad = k + padding;             // 2049 instead of 2048
uint32_t size_pad = k * k_pad;
uint32_t size     = k * k;

dtype *src = (dtype*)aligned_alloc(sizeof(dtype), sizeof(dtype) * size_pad);
dtype *dst = (dtype*)aligned_alloc(sizeof(dtype), sizeof(dtype) * (size_pad + 1));
```

At first, let's consider `padding = 1`.
Two things changed. The stride is now 2049, giving a row stride of $2049 \times 8 = 16392$ bytes. This is *not* a multiple of $64 \times 64 = 4096$: we get $16392 \bmod 4096 = 8$, meaning each row shifts by one cache line relative to the previous in set-index space. Over the 8 rows of a Z-order block, the accesses now spread across 8 different sets rather than colliding in one. The resonance is broken.

Also, `src` and `dst` are now returned by separate `aligned_alloc` calls, so they live in independent heap regions rather than being carved from a single block. This helps too.

The loop is the same, just using `k_pad` as the stride:

```
for (uint32_t counter = 0; counter < size; counter++) {
    coord_t c = compact_bits(counter);
    uint32_t src_pos = c.x * k_pad + c.y;
    uint32_t dst_pos = c.y * k_pad + c.x;
    dst[dst_pos] = src[src_pos];
}
```

Still k^2 iterations — we transpose the logical $k \times k$ matrix, stored with padded stride.

---

6. The measurements

Both versions: same compiler flags (`gcc -O3 -march=native -mtune=native`), same matrix (2048×2048 doubles), same machine, pinned to P-core 0. Per-repetition averages:

Counter	v1 (no padding)	v2 (padded)	Ratio
`PAPI_TOT_INS`	201,326,844	40,894,740	**4.92×**
`PAPI_TOT_CYC`	50,908,049	26,251,244	1.94×
`PAPI_L1_LDM`	1,066,236	1,017,522	1.05×
`PAPI_L1_STM`	780,731	834,684	0.94×
`PAPI_LD_INS`	4,194,448	4,718,740	0.89×
`PAPI_SR_INS`	4,194,389	4,194,392	1.00×

The cache numbers are roughly what I expected. Load miss rate: 25.4% (v1) vs 21.6% (v2). Store miss rate: 18.6% vs 19.9%. Both higher than the ideal 12.5%, not hugely different from each other. The padding helped the wall time — v2 is about 2× faster in cycles — which probably means the out-of-order engine overlaps misses with computation better when the set-conflict pattern is less degenerate. Shorter stall chains, less serial waiting. Fine.
NOTE: *When testing larger padding, for instance* `65`*, the* `L1 miss` *ratio both in read and in write gets back to the expected* 12.5%.

What I did *not* expect was the instruction count.

v1: 201 million. v2: 41 million. Five to one.

And yet loads and stores are nearly identical: ~4.2 million each, which is exactly $2048^2 = 4{,}194{,}304$. One load and one store per element, as it should be. The entire factor-of-five gap is in non-memory instructions — the shifts, ANDs, XORs of `compact_bits`, the address arithmetic, the loop overhead. The two versions are executing *fundamentally different machine code* for what looks like the same C source.

The IPC confirms this. v1 runs at ≈ 3.95, which is essentially the retirement width of Golden Cove — the core is saturated with simple ALU operations, no stalls, the signature of code dominated by integer bit manipulation. v2 runs at ≈ 1.56, typical of memory-bound code where the compute is lean and the bottleneck is waiting for data.

I was supposed to be studying cache effects. Instead I had walked into a vectorisation problem.

---

7. The assembly

Since the compiler flags were identical, the answer had to be in the generated code. I added `-S -fverbose-asm` and went to look.

7.1. Version 1 — scalar

The transpose loop in v1 compiles to a tight scalar loop at label `.L33`. Here is the core of it:

```
.L33:
    ; --- compact_bits inlined, scalar ---
    mov     eax, esi            ; a = counter
    mov     ecx, esi            ; b = counter
    add     esi, 1              ; counter++
    and     eax, 1431655765     ; a &= 0x55555555
    shr     ecx                 ; b >>= 1
    mov     edx, eax
    and     ecx, 1431655765     ; b &= 0x55555555
    shr     edx
    xor     edx, eax
    ; ... 8 more shift/xor/and pairs ...
    movzx   edx, dx             ; a &= 0xFFFF
    movzx   eax, ax             ; b &= 0xFFFF

    ; --- address computation ---
    shlx    ecx, edx, r14d      ; c.x * k  (shift, k is power of 2)
    add     ecx, eax            ; src_pos
    shlx    eax, eax, r14d      ; c.y * k
    add     eax, edx            ; dst_pos

    ; --- one load, one store ---
    vmovsd  xmm0, [r15+rcx*8]
    vmovsd  [rbx+rax*8], xmm0

    cmp     r13d, esi
    jne     .L33
```

About 48 instructions per iteration, all scalar. The function is inlined but processes one element at a time. The only SIMD in sight is the `vmovsd` pair, which is just the x86-64 way of moving a double. Over $4{,}194{,}304$ iterations: ≈ 201 million instructions. Matches PAPI.

7.2. Version 2 — AVX2

The inner loop in v2 (label `.L25`) is way different:

```
.L25:
    vmovdqa ymm0, ymm4              ; 8 packed counters
    add     eax, 1
    vpaddd  ymm4, ymm4, [rbp-272]   ; counters += 8

    ; --- compact_bits vectorised ×8 ---
```

```
    vpand   ymm1, ymm0, ymm9          ; a = counter & 0x55555555
    vpsrld  ymm0, ymm0, 1             ; b = counter >> 1
    vpand   ymm0, ymm0, ymm9          ; b &= 0x55555555
    vpsrld  ymm3, ymm1, 1
    vpxor   ymm3, ymm3, ymm1
    vpand   ymm3, ymm3, ymm8          ; (a^(a>>1)) & 0x33333333
    ; ... remaining stages, all on ymm ...
    vpand   ymm1, ymm1, ymm5          ; 8 x-coords
    vpand   ymm0, ymm0, ymm5          ; 8 y-coords

    ; --- address computation ×8 ---
    vpmulld ymm3, ymm1, ymm10         ; c.x * k_pad
    vpaddd  ymm3, ymm3, ymm0          ; + c.y
    vpmulld ymm0, ymm0, ymm10         ; c.y * k_pad
    vpaddd  ymm0, ymm0, ymm1          ; + c.x

    ; --- manual gather ---
    vpmovzxdq  ymm11, xmm3
    vmovq   rdx, xmm11
    mov     rdi, [r12+rdx*8]          ; load src[idx0]
    vpextrq rdx, xmm11, 1
    mov     rcx, [r12+rdx*8]          ; load src[idx1]
    ; ... 6 more scalar loads ...

    ; --- manual scatter ---
    vmovq   rdx, xmm1
    mov     [rbx+rdx*8], rdi
    vpextrq rdx, xmm1, 1
    mov     [rbx+rdx*8], rcx
    ; ... 6 more scalar stores ...

    jne     .L25
```

Eight elements per iteration. The entire `compact_bits` computation and address arithmetic are vectorised — `vpsrld`, `vpand`, `vpxor`, `vpmulld` all operating on 8 packed 32-bit integers in 256-bit ymm registers.

The loads and stores, on the other hand, cannot be vectorised — the indices are non-contiguous, it is a scatter/gather pattern — so GCC extracts each index individually and does scalar memory accesses. This is the bottleneck of the vectorised version, but the 8× reduction in compute instructions more than pays for it.

About 75–80 instructions per vector iteration, for 8 elements. Over $524{,}288$ iterations: ~40 million. Matches the measurement.

Small detail: the address computation uses `vpmulld` (packed multiply) rather than a shift, because $k_{\text{pad}} = 2049$ is not a power of two. But `vpmulld` has a throughput of one per cycle on Golden Cove, so amortised over 8 elements it costs practically nothing.

7.3. The aliasing explanation

So: same source code, same flags, one is scalar, the other is AVX2. Why?

The answer is pointer aliasing and laziness.

In v1:

```
dtype *src = (dtype*)malloc(sizeof(dtype) * size * 2);
dtype *dst = src + size;
```

Both pointers refer to the same allocation. In practice they do not overlap — `src` is the first half, `dst` the second — but the compiler does not know this. The Z-order indices come from `compact_bits`, which is, from the compiler's perspective, an opaque bit-manipulation; `src_pos` and `dst_pos` could be anything. GCC cannot establish that `dst[dst_pos]` never coincides with `src[src_pos]` for some combination of iterations. It cannot prove that processing 8 iterations in parallel (which is what vectorisation means) would produce the same result as doing them one at a time. A vectorised store might overwrite something that a later load in the same batch still needs.

So GCC's alias analysis gives up and falls back to scalar. Correctness first.

In v2:

```
dtype * restrict src = (dtype*)aligned_alloc(sizeof(dtype), sizeof(dtype) * size_pad);
dtype * restrict dst = (dtype*)aligned_alloc(sizeof(dtype), sizeof(dtype) * (size_pad +
1));
```

Two separate allocation calls. The C standard guarantees that distinct `malloc`/`aligned_alloc` calls return non-overlapping memory, and GCC's alias oracle knows this. It can prove that writes through `dst` never interfere with reads through `src`, no matter what the indices do. Vectorisation is safe; GCC takes it.

One `malloc` versus two. That is the whole difference. 201 million instructions versus 41 million.

7.4. Verification

To confirm, I've add `restrict` to the v1 pointers and split them into two separate call (see the file `matrix_transpose_z-order.v1.fixed.c`):

```
dtype * restrict src = (dtype*)aligned_alloc(sizeof(dtype), sizeof(dtype) * size);
dtype * restrict dst = (dtype*)aligned_alloc(sizeof(dtype), sizeof(dtype) * size);
```

This promises the compiler that `src` and `dst` do not alias. GCC should then vectorise v1 and the instruction count should drop to ~41 million.

On the other hand, you can break v2's vectorisation by going back to a single allocation:

```
dtype *buf = aligned_alloc(sizeof(dtype), sizeof(dtype) * (2 * size_pad + 1));
dtype *src = buf;
dtype *dst = buf + size_pad;
```

Without `restrict`, the instruction count should jump to ~200 million again.

Looking back, I realised that the divide-et-impera code from Section 2.1 had been doing the right thing from the start, and for two reasons. It allocates `src` and `dst` with two separate `malloc` calls, so the compiler can prove non-aliasing; and `src` is declared `const dtype *`, which explicitly guarantees the source is never written. I did not pay attention to any of this when I wrote that code — it was just what seemed like a correct and clean way to write it.
Turns out that "correct and clean" had performance consequences I had not anticipated, although I should have seen it by design.

---

8. Lessons, such as they are

I started this exercise to prepare a clean example of how elegant algorithm may result to be unpractical (due to instructions bloat) or have side effects (like the cache set conflicts; this second popped out while working, to be honest).
What I got was something more tangled and, in the end, more useful. A few observations, not in any particular order.

Cache *geometry* matters as much as cache *size*. The resonance problem has nothing to do with capacity — the matrix is a thousand times larger than L1 regardless. It is about the mapping to sets. When the stride resonates with the number of sets, the effective associativity collapses, and the cache is essentially switched off. For power-of-two dimensions this happens reliably.

Padding works and costs nothing (but memory). One extra element in the leading dimension breaks the resonance completely. Every serious linear algebra library does this. If you have power-of-two array dimensions, pad them. Just do it.

Keep your counters separate. The `PAPI_L1_DCM` trap — mixing load and store misses — is easy to fall into and hard to notice once you are in it. The ratio looks plausible enough that you can stare at it for a while before realising it is nonsensical.

Always profile; never guess. Wall-clock time showed a 2× difference between v1 and v2. There are a hundred possible explanations for a factor of two. The counters pointed to the real one: the cache behaviour was similar; the instruction count was wildly different. Without PAPI, I would have attributed the speedup to better caching and moved on — confident, and completely wrong.

Aliasing is a compiler-visible property that has nothing to do with what the code actually does. One `malloc` versus two changes nothing about the algorithm, nothing about the data layout, nothing about the cache behaviour. It changes what the compiler can *prove*, and that was worth a factor of five in code density. The gap between a 48-instruction scalar body and a 10-instruction-per-element vectorised loop came down to an allocation decision I made without thinking.

And finally: look at the assembly. I know it sounds tedious, and it sometimes is, but when two versions of what you believe is the same code differ by 5× in instruction count, the answer is not in the source — it is in what the compiler actually produced. `-S -fverbose-asm` is not heroic; it is the minimum.

The most important lesson, though, is about benchmarking itself. I set out to measure one thing (cache conflicts) and accidentally changed something orthogonal (vectorisation). The PAPI counters caught the accident. Wall-clock time alone would not have. If there is one thing I want you to take away from these notes, it is this: performance measurement is harder than it looks, and you should instrument everything you can, because the numbers will surprise you — and the surprises are where you actually learn something.

Appendix: compilation and execution

```
# v1 — single allocation
gcc -O3 -march=native -mtune=native -DDTYPE=double -DUSE_PAPI \
    -o transpose_v1 matrix_transpose_z-order_v1.c -lpapi

# v2 — padded stride, separate allocations
gcc -O3 -march=native -mtune=native -DDTYPE=double -DUSE_PAPI \
    -o transpose_v2 matrix_transpose_z-order_v2.c -lpapi

# run, 2048×2048 matrix, pinned to P-core 0
taskset -c 0 ./transpose_v1 11
taskset -c 0 ./transpose_v1.fixed 11
taskset -c 0 ./transpose_v2 11 $padding

# assembly
gcc -O3 -march=native -mtune=native -DDTYPE=double -DUSE_PAPI \
    -S -fverbose-asm -o transpose_v1.s matrix_transpose_z-order_v1.c
gcc -O3 -march=native -mtune=native -DDTYPE=double -DUSE_PAPI \
    -S -fverbose-asm -o transpose_v2.s matrix_transpose_z-order_v2.c
```